Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное учреждение
высшего образования «Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Отчёт по лабораторной работе № 2

по дисциплине «Операционные системы»

LNX-4 VTKM: вывод `struct task_struct` через системный вызов

(демонстрационное решение)

Выполнил:
студент группы P3331
Данилов Тимофей Николаевич

Преподаватель практики:
Максимов Андрей Николаевич

Санкт-Петербург, 2026

1 Задание

Программа пользователя получает аргумент (не адрес), передаёт его в ядро и печатает поля найденной ядром структуры. Загружаемый модуль ядра принимает запрос, находит структуру через системные таблицы и возвращает данные.

Вариант: интерфейс **syscall**, структура задаётся именем из заголовков Linux. Выбрана `struct task_struct`, аргумент — PID.

2 Решение

```
./vtkm-cli PID
   │  syscall(549, pid, &info, sizeof info)
   ▼
ядро: SYSCALL_DEFINE3(vtkm)      заглушка, вшита в ядро патчем
   │  вызывает обработчик модуля; без модуля -ENOSYS
   ▼
vtkm.ko: PID → struct pid → task_struct → struct vtkm_info → copy_to_user
```

Почему патч ядра. Модуль не может сам добавить системный вызов: `sys_call_table` не экспортируется, а x86-64 с версии 6.9 выбирает обработчик через `switch` в `x64_sys_call`, собранный при компиляции ядра. Поэтому в ядро (Linux 6.18) добавлены номер 549 и заглушка, а вся логика осталась в модуле, который регистрирует свой обработчик при загрузке.

2.1 Заглушка в ядре

```
SYSCALL_DEFINE3(vtkm, int, pid, void __user *, buf, size_t, size)
{
  long ret = -ENOSYS;

  mutex_lock(&vtkm_lock);
  if (vtkm_handler)
    ret = vtkm_handler(pid, buf, size);
  mutex_unlock(&vtkm_lock);
  return ret;
}
```

Модуль ставит обработчик через `vtkm_register` и снимает через `vtkm_unregister`. Мьютекс держится и во время вызова, поэтому `rmmod` ждёт, пока текущие вызовы выйдут из модуля, и код модуля не выполняется после выгрузки.

2.2 Модуль

```
  if (size != sizeof(info))
    return -EINVAL;
  p = find_get_pid(nr);                /* таблица PID */
```

```
  if (p == NULL)
    return -ESRCH;
  task = get_pid_task(p, PIDTYPE_PID);  /* +1 ссылка на task_struct */
  put_pid(p);
  if (task == NULL)
    return -ESRCH;

  memset(&info, 0, sizeof(info));
  fill_info(task, &info);               /* comm, pid, tgid, ppid, state, prio, ... */
  put_task_struct(task);

  if (copy_to_user(buf, &info, sizeof(info)) != 0)
    return -EFAULT;
```

Наружу копируется своя структура `vtkm_info` (общий заголовок модуля и программы) только со значениями полей, без указателей ядра. `total_vm` берётся из `mm_struct` через `get_task_mm`/`mmput`; у потоков ядра `mm` нет, поле остаётся 0.

Программа `vtkm-cli` проверяет аргумент, делает `syscall` и печатает поля; на `ENOSYS` подсказывает загрузить модуль, на `ESRCH` пишет, что процесса нет.

3 Проверка

Окружение: виртуальная машина `vtkm` (Ubuntu 24.04, ядро 6.18 с патчем).

`./vtkm-cli 1`

```
struct task_struct of PID 1
  comm         systemd
  pid          1
  tgid         1
  ppid         0
  state        S
  prio         120
  static_prio  120 (nice 0)
  policy       0 (SCHED_OTHER)
  threads      1
  utime        693000000 ns
  stime        1130601219 ns
  nvcsw        4524
  nivcsw       910
  total_vm     5503 pages (22012 kB)
```

`/proc/1/status`

```
Name: systemd
State:  S (sleeping)
Tgid: 1
Pid:  1
PPid: 0
VmSize:    22012 kB
Threads:  1
voluntary_ctxt_switches:  4524
nonvoluntary_ctxt_switches: 910
```

Значения совпадают с `/proc/1/status`, в том числе переключения контекста (4524, 910) и память: 5503 страницы × 4 КиБ = 22012 kB = `VmSize`.

Скрипт `test.sh` проверяет PID 1, свой процесс, `nice`, поток ядра, PID потока, ошибки и работу без модуля:

```
OK   без модуля: ENOSYS и подсказка
OK   PID 1: comm
OK   PID 1: ppid
OK   PID 1: state
OK   sleep: comm sleep
OK   sleep: ppid = этот скрипт
OK   sleep: state S
OK   nice 10: static_prio 130
OK   PID 2: kthreadd, total_vm 0
OK   поток 2836: pid 2836, tgid 2835
OK   нет такого PID: код 1
OK   буквы вместо PID: код 2
OK   без аргумента: код 2
OK   PID 0: код 2
OK   после rmmod: код 1
passed 15, failed 0
```

4 Выводы

Системный вызов реализован как заглушка в ядре и обработчик в загружаемом модуле: так модуль можно пересобирать без пересборки ядра, а выгрузка безопасна благодаря мьютексу. Задача ищется через таблицу PID с подсчётом ссылок, пользователю передаются только значения полей через `copy_to_user`. Вывод совпадает с `/proc/<pid>/status`, все проверки проходят.

Код: репозиторий `~/projects/vtkm-example` в ВМ `vtkm`, порядок работы — в истории коммитов.

5 Использование LLM

Код этого демонстрационного решения и отчёт сгенерированы Claude Opus 5.5 (Claude Code) по просьбе студента как пример выполнения работы. По правилам курса ключевую логику нельзя писать через LLM, поэтому решение используется только для разбора и не сдаётся как собственная работа.